Tim Pengajar IF2150

IF2150 – Rekayasa Perangkat Lunak

Domain Specific Language

SEMESTER I TAHUN AJARAN 2025/2026

KNOWLEDGE & SOFTWARE ENGINEERING

General Purpose Language

- Turing complete
- Well understood and widely used
- Applicable to a wide range of problems

Problem Solving using GPL

- Analyze the problem domain
- Express the conceptual model as an OO/FP/... Design
- Program/develop a framework/library
- Express concrete application as framework/library client
- Pros:
  - predictable and familiar result
  - (relatively) low cost of implementation
- Cons:
  - difficult to fully exploit domain-specific knowledge
  - only available to general programmers

GPL Pros

- Excellent support through IDEs
- Easy to find experienced developers
- Growth through libraries

GPL Cons

- Growth of language is (often) not possible
- Can be very verbose
- Lack of abstractions

Domain Specific Language

- Small language targeted at a particular problem domain
- Expressive in its own domain
- Often declarative

Problem Solving using DSL

- Analyze the problem domain
- Express the conceptual model as a language design
- Implement a compiler or interpreter
- Pros:
  - possible to exploit all domain-specific knowledge
  - also available to domain experts
- Cons:
  - (relatively) high cost of implementation
  - risk of Babylonian confusion
  - lack of tool support (IDE,...)
  - hard to combine DSLs or DSL and GPL this way

DSL Pros

- Tailored to a particular application domain [Mern05a]
- Expressive and easy to use [Huda98a]
- Little languages, little maintenance, increased productivity [Deur97a]
- Communication with domain experts [Fowl08X]

DSL Cons

- Extra investment
  - Language engineering and maintenance
  - Learning a new language
- Weak IDE/Tool support
  - Editor, Debugger, Refactoring
- Migration might be difficult
- Evolving into generality

Programming languages:

PADS, Datascript: Data description

scientific computation

HTML: Markup Documents

Fortran

Cryptol: Cryptography

R-anguage: Statitstics

business applications

Envision: Computer vision

systems programming

awk, sed, find: O/S toolkit

XQuery: Querying XML

Lex/YACC: Parser generation

compiler construction

makefiles: Application construction

ML

Perl

autoconf: System configuration

Simula

SQL: Querying relational data

web

user-interfaces

XSLT: Transforming XML

Lisp

Latex: Typesetting

Postscript: Printing

symbolic computation

and many more…

Why do we need a language

- Languages supply a rich interface to computers.

```
#!/sw/bin/ruby
require 'uri'; require  'net/http'

uri= URI.parse(ARGV[0])
h=Net::HTTP.new(uri.host,80)

resp,data = h.get(uri.path)
hwk = {}
if resp.message == "OK"
   data.scan(/Homework (\d*) \(due (\d*)\/(\d*)\)/)
                                    {|x,y,z| hwk[x] = Time.local(2005,y,z)}

end

hwk.each{| assignment, duedate|
  if duedate < (Time.now - 60 * 60 * 24)
     puts "Hwk #{assignment} was due on #{duedate.strftime("%A, %B %d")}."
  else
     puts "Hwk #{assignment} is due on #{duedate.strftime("%A, %B %d")}."
  end
 }
```

vs

- Languages directly provide a model of the computational domain.

Rationale of using DSLs

Tailored abstraction

- Increase accessibility for domain experts
- Improve reliability
  - Programs are shorter.
  - Compiler generates tedious boilerplate code.
- Allow programs to serve as "living documentation"

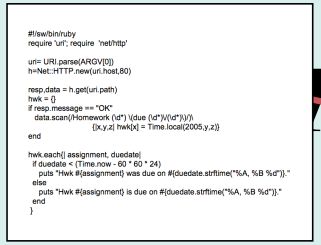
Rationale (2)

Restricting expressiveness

It enables validation and optimization at domain-level.

- SQL programs are guaranteed to terminate.
- YACC specifications are guaranteed to compile into PDAs.
- Cryptol programs are guaranteed to require only finite space.

DSL for various purposes



```
#!/sw/bin/ruby
require 'uri'; require  'net/http'

uri= URI.parse(ARGV[0])
h=Net::HTTP.new(uri.host,80)

resp,data = h.get(uri.path)
hwk = {}
if resp.message == "OK"
   data.scan(/Homework (\d*) \(due (\d*)/\(\d*)\)/)\
                        {|x,y,z| hwk[x] = Time.local(2005,y,z)}
end

hwk.each{| assignment, duedate|
  if duedate < (Time.now - 60 * 60 * 24)
    puts "Hwk #{assignment} was due on #{duedate.strftime("%A, %B %d")}."
  else
    puts "Hwk #{assignment} is due on #{duedate.strftime("%A, %B %d")}."
  end
}
```

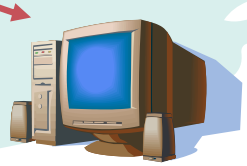
Rat *(3)*

DSL

Verification support

Special purpose hardware

Executable



Auxiliary tools

C/Java libraries

Rationale (4)

- Dramatically shorter code
- Description is short enough to serve as documentation.
- **Safer**: error code inserted automatically and completely (as long as the is compiler right…).
- **Leverage**: produce value-added tools.

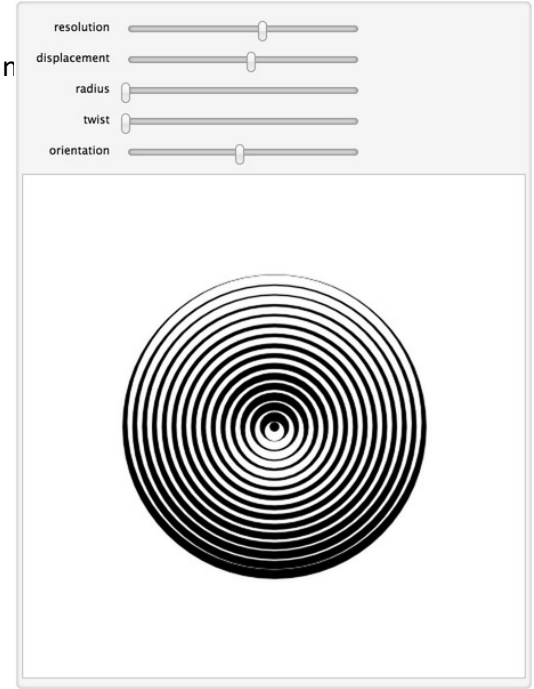
HTML

```html
<!DOCTYPE html>
<html>
    <head>
        <title>Example<title>
        <link rel="stylesheet" href="styl
    </head>
    <body>
        <h1>
            <a href="/">Header</a>
        </h1>
        <nav>
            <a href="one/">One</a>
            <a href="two/">Two</a>
            <a href="three/">Three</a>
        </nav>
```

KNOWLEDGE & SOFTWARE ENGINEERING

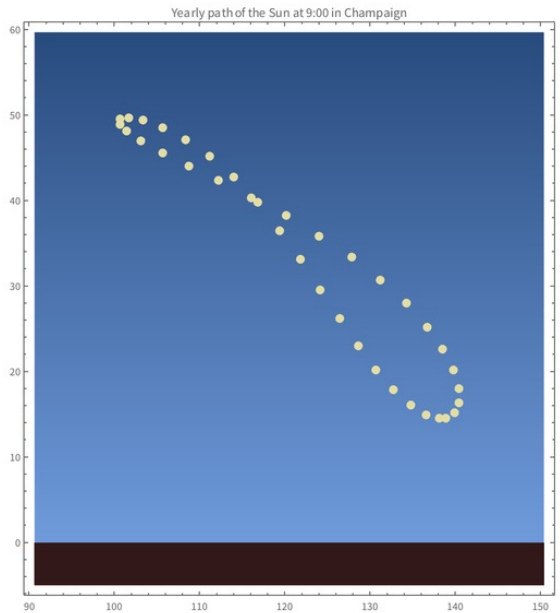
Wolfram Language

```
LogoDesignWidget[] :=
 Manipulate[
  Graphics[
   Rotate[
    Table[Rotate[
      {
       If[EvenQ[i], Black, White],
       Disk[{0, If[EvenQ[i], 0, Rescale[i, {0, n}, (d/n
        Rescale[i, {0, n}, {1, r}]]
       },
      2 \[Pi] tw (i/n), {0, 0}
      ], {i, 0, n}],
    or, {0, 0}
    ], PlotRange -> 1.5, ImageSize -> 400],
  {{n, 31, "resolution"}, 1, 51},
  {{d, 1, "displacement"}, -10, 10},
  {{r, 0, "radius"}, 0, 1},
  {{tw, 0, "twist"}, 0, 10},
  {{or, 0, "orientation"}, -\[Pi], \[Pi]}
  ]
```

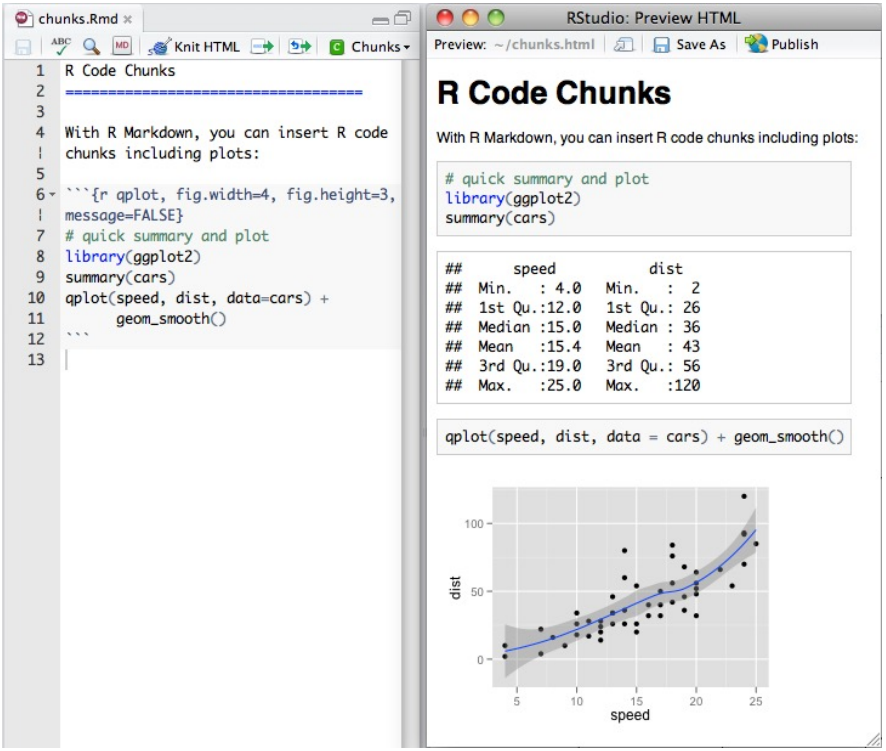


Plot the Yearly Path of the Sun

```
SunPlot[] :=
 Block[{points, xMin, xMax, yMin, yMax,
   skyTopColor = Hue[0.6, 0.7, 0.5],
   skyBottomColor = Hue[0.6, 0.5, 0.9],
   groundColor = Hue[0, 0.5, 0.2], sunColor = Hue[0.15, 0.3, 0.9]},
  points = Map[QuantityMagnitude, SunPosition[DateRange[
       DateObject[{2014, 1, 1, 9, 0}],
       DateObject[{2014, 12, 31, 9, 0}], 10]]["Values"], {2}];

  {{xMin, xMax}, {yMin, yMax}} = ({Min[#], Max[#]} + 10 {-1, 1}) & /@
    Transpose[points];
  yMin = -5;

  Graphics[{
    {Blue,
     Polygon[{{xMin, yMin}, {xMin, yMax}, {xMax, yMax}, {xMax,
        yMin}},
      VertexColors -> {skyBottomColor, skyTopColor, skyTopColor,
        skyBottomColor}]},
    {groundColor, Rectangle[{xMin, yMin}, {xMax, 0}]},
    {sunColor, Disk[#, .5] & /@ points}
    }, Frame -> True, ImageSize -> 500,
   PlotLabel ->
    "Yearly path of the Sun at 9:00 in " <>

     CommonName[First[GeoIdentify["City", $GeoLocation]]]]
  ]
```



R language


chunks.Rmd
Knit HTML
Chunks
R Code Chunks
==========
With R Markdown, you can insert R code chunks including plots:
```{r qplot, fig.width=4, fig.height=3, message=FALSE}
# quick summary and plot
library(ggplot2)
summary(cars)
qplot(speed, dist, data=cars) +
geom_smooth()
```
RStudio: Preview HTML
Preview: ~/chunks.html
Save As
Publish
R Code Chunks
With R Markdown, you can insert R code chunks including plots:
# quick summary and plot
library(ggplot2)
summary(cars)
speed dist
Min. : 4.0 Min. : 2
1st Qu.:12.0 1st Qu.: 26
Median :15.0 Median : 36
Mean :15.4 Mean : 43
3rd Qu.:19.0 3rd Qu.: 56
Max. :25.0 Max. :120
qplot(speed, dist, data = cars) + geom_smooth()
dist
speed

COBOL

```
000010 IDENTIFICATION DIVISION.
000020 PROGRAM-ID.       SAMPLE.
000030 AUTHOR.           J.P.E. HODGSON.
000040 DATE-WRITTEN.     4 February 2000
000041
000042* A sample program just to show the form.
000043* The program copies its input to the output,
000044* and counts the number of records.
000045* At the end this number is printed.
000046
```

```
00130 DATA DIVISION.
000140 FILE SECTION.
000150 FD  STUDENT-FILE
000160     RECORD CONTAINS 43 CHARACTERS
000170     DATA RECORD IS STUDENT-IN.
000180 01  STUDENT-IN              PIC X(43).
000190
000200 FD  PRINT-FILE
000210     RECORD CONTAINS 80 CHARACTERS
000220     DATA RECORD IS PRINT-LINE.
000230 01  PRINT-LINE              PIC X(80).
```

KNOWLEDGE & SOFTWARE ENGINEERING

```
PROCEDURE DIVISION.

PREPARE-SENIOR-REPORT.
    OPEN INPUT  STUDENT-FILE
         OUTPUT PRINT-FILE.
    MOVE ZERO TO RECORDS-WRITTEN.
    READ STUDENT-FILE
        AT END MOVE 'NO' TO DATA-REMAINS-SWITCH
    END-READ.
    PERFORM PROCESS-RECORDS
        UNTIL DATA-REMAINS-SWITCH = 'NO'.
    PERFORM PRINT-SUMMARY.
    CLOSE STUDENT-FILE
          PRINT-FILE.
    STOP RUN.
```

```
PROCESS-RECORDS.
    MOVE STUDENT-IN TO RECORD-IMAGE.
    MOVE DETAIL-LINE TO PRINT-LINE.
    WRITE PRINT-LINE.
    ADD 1 TO RECORDS-WRITTEN.
    READ STUDENT-FILE
        AT END MOVE 'NO' TO DATA-REMAINS-
SWITCH
    END-READ.

PRINT-SUMMARY.
    MOVE RECORDS-WRITTEN TO TOTAL-READ.
    MOVE SUMMARY-LINE TO PRINT-LINE.
    WRITE PRINT-LINE.
```

SQL

Language for querying relational data bases.

Students

ID	NAME
01	Harry Potter
02	Hermione Granger
03	Ronald Weasley

Potions

ID	GRADE
01	Satisfactory
02	Outstanding
03	Satisfactory

```
SELECT Students.NAME,
       Potions.GRADE
FROM Students, Potions
WHERE Students.ID = Potions.ID
```

NAME	GRADE
Harry Potter	Satisfactory
Hermione Granger	Outstanding
Ronald Weasley	Satisfactory

KNOWLEDGE & SOFTWARE ENGINEERING

SQL

- SQL compiles into relational algebra with select, project, and join logical operators.
- Query engine chooses corresponding physical operators based on indices and other statistics about the data.
- Years of research have gone into the best query plan selection and join algorithms.
- Data analyst can be blissfully ignorant of details under the covers.

DSL in Financial Domain

Financial Domain-Specific Language Listing

The goal of this page is to provide a comprehensive listing of financial domain-specific languages and resources. Please submit additional resources via GitHub.

Actulus Modeling Language (AML)

The goal of the Actulus project is to "establish a platform for definition of advanced life insurance and pension products and for efficient computations on them." Actulus is a joint effort between the the IT University of Copenhagen and Edlund A/S. The project uses QuantAlea (see below) for GPU kernel creation.

- An Actuarial Programming Language for Life Insurance and Pensions. David R. Christiansen, Klaus Grue, Henning Niss, Peter Sestoft, Kirstjan S. Sigtryggsson. 2013.
- Domain-specific languages and GPGPUs in life insurance and pensions. Peter Sestoft. 2015.
- GPUs and DSLs for Life Insurance Modeling. Peter Sestoft. 2016.

Alea.cuBase (by QuantAlea)

Alea.cuBase is an F# framework for generating GPU kernels. It uses F# quotations for staging and composing kernels at run-time. It provides a "foundation to develop GPU aware domain specific languages."

- Dynamic CUDA with F#. Dr. Daniel Egloff and Xiang Zhang. GPU Technology Conference 2013.

AxLang (by Axoni)

AxLang is a DSL embedded in Scala for writing verifiable smart contracts for Ethereum. The language includes pre- and post-conditions to support design-by-contract-style verification.

- Announcement Blog Post
- Announcement YouTube video

BOScoin Trust Contracts

BOScoin uses the Web Ontology Language (OWL) and Timed Automata Language (TAL) to encode contracts on a blockchain.

CPL: Chart Pattern Language

- Charting Patterns on Price History. Saswat Anand, Wei-Ngan Chin and Siau-Cheng Khoo. ICFP '01: International Conference on Functional Programming.

http://www.dslfin.org/resources.html

Cryptol

- A language, developed at Galois, for expressing cryptographic algorithms.
- More information:

http://www.cryptol.net

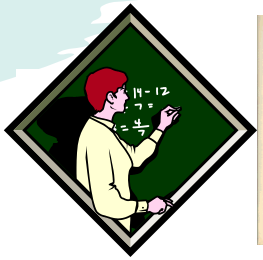
Cryptol
The Language of Cryptography

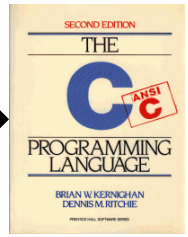
Thanks to Galois for contributing material for these slides.

Crypto-algorithm correctness

- Verification and validation a critical piece of crypto-modernization programs.
- Exploding complexity and requirements
  - Number of algorithms, hardware platforms
  - High assurance requirements
- 25% of algorithms submitted for FIPS validation had flaws (according to Director of NIST CMVP, 2002)

Why is this hard?

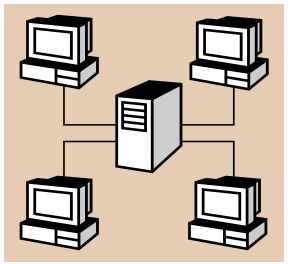




Requires skills in math AND programming

Variety of target architectures

Validation is complex and tedious



Variety of requirements

Lack of clear reference implementations

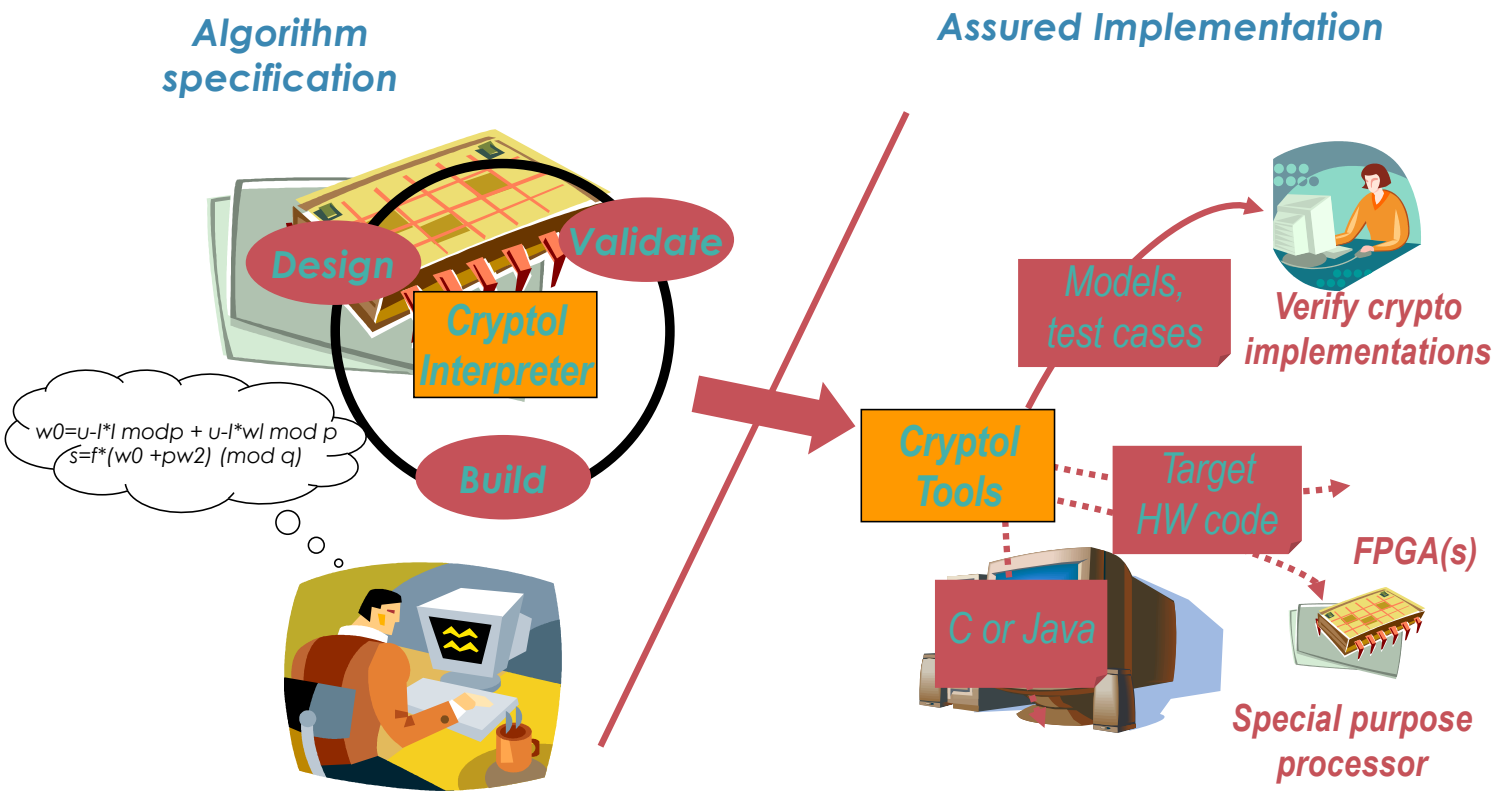
$$
\begin{aligned}
x_i &= \lfloor X/2^{8i} \rfloor \bmod 2^8 \qquad i = 0, \ldots, 3 \\
y_i &= s_i[x_i] \qquad i = 0, \ldots, 3 \\
\begin{pmatrix} z_0 \\ z_1 \\ z_2 \\ z_3 \end{pmatrix} &= \begin{pmatrix} \cdot & \cdots & \cdot \\ \vdots & \text{MDS} & \vdots \\ \cdot & \cdots & \cdot \end{pmatrix} \cdot \begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \end{pmatrix} \\
Z &= \sum_{i=0}^{3} z_i \cdot 2^{8i}
\end{aligned}
$$

```
#define MDS_GF_FDBK     0x169
#define LFSR1(x)  ( ((x) >> 1)  ^ (((x) & 0x01) ?
MDS_GF_FDBK/2 : 0))
#define LFSR2(x)  ( ((x) >> 2)  ^ (((x) & 0x02) ?
                    MDS_GF_FDBK/2 : 0)
                    ^ (((x) & 0x01) ?
                    MDS_GF_FDBK/4 : 0))
#define Mx_1(x)  ((DWORD)   (x))
#define Mx_X(x)  ((DWORD)  ((x) ^ LFSR2(x)))
#define Mx_Y(x)  ((DWORD)  ((x) ^ LFSR1(x) ^
LFSR2(x)))
#define M00             Mul_1
#define M01             Mul_Y
return  ((M00(b[0]) ^ M01(b[1]) ^
         M02(b[2]) ^ M03(b[3]))          ) ^
        ((M10(b[0]) ^ M11(b[1]) ^
                                        ^
        ((M20(b[0]) ^ M21(b[1]) ^
         M22(b[2])^ M23(b[3])) <<16)^
        ((M30(b[0]) ^ M31(b[1]) ^
         M32(b[2])^ M33(b[3])) <<24);
```

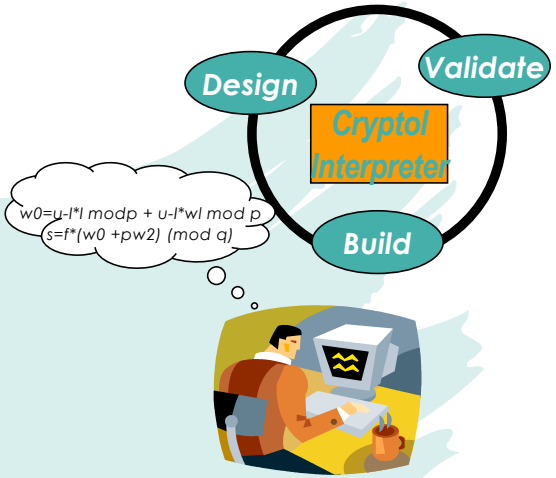
It's hard to relate implementations to the underlying math

KNOWLEDGE & SOFTWARE ENGINEERING

One Specification - Many Uses



Algorithm specification

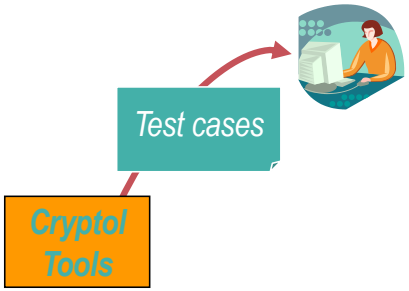


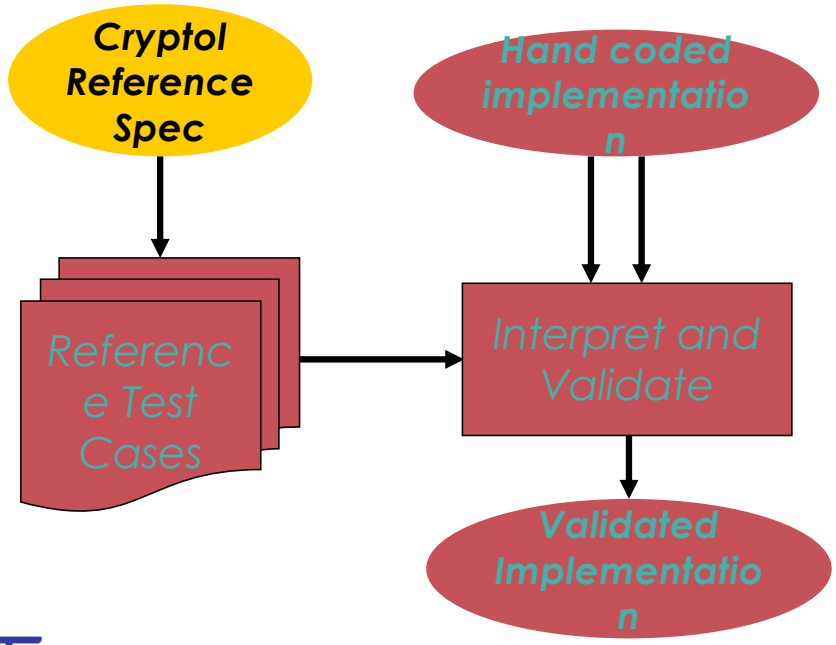

```
rc6ks : {a} (w >= width a) =>
        [a][8] -> [r+2][2][w];
rc6ks key = split (rs >>> (v - 3 * nk))
  where {
    c = max (1, (width key + 3) / (w / 8));
    v = 3 * max (c, nk);
    initS =  [pw (pw+qw) ..]@@[0 .. (nk-1)];
    padKey : [4*c][8];
    padKey = key # zero;
    initL : [c][w];
    initL = split (join padKey);
    ss = [| (s+a+b) <<< 3
            || s <- initS # ss
            || a <- [0] # ss
            || b <- [0] # ls |];
    ls = [| (l+a+b) <<< (a+b)
            || l <- initL # ls
            || a <- ss
            || b <- [0] # ls |];
    rs = ss @@ [(v-nk) .. (v-1)];
  };
```

- Models crypto-algorithm
- Natural expression
- Clear and unambiguous
- Structure and guide an implementation

KNOWLEDGE & SOFTWARE ENGINEERING

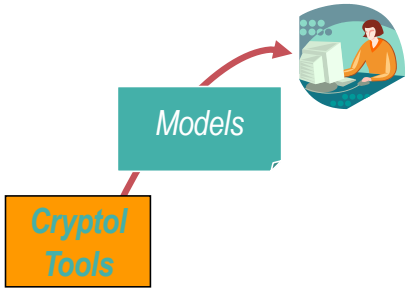
Usage: Testing

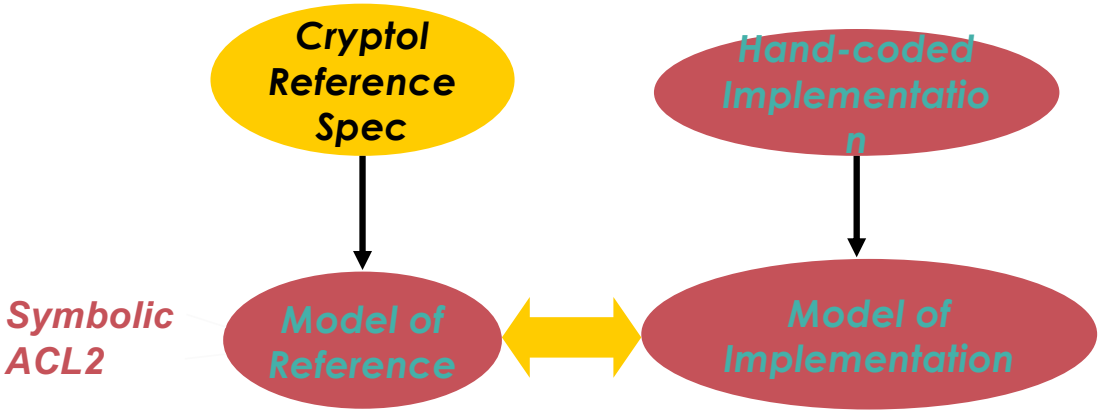




- Generates "known good tests"
- Built-in capture of intermediate vectors simplifies debugging
- Easy to generate new intermediate vectors as needed

Usage: Verification

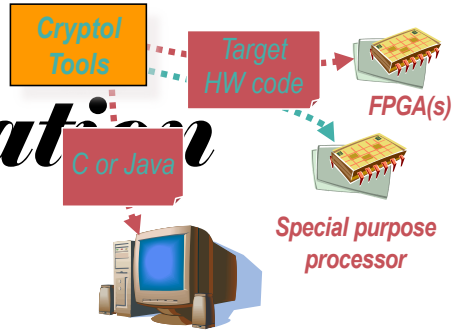




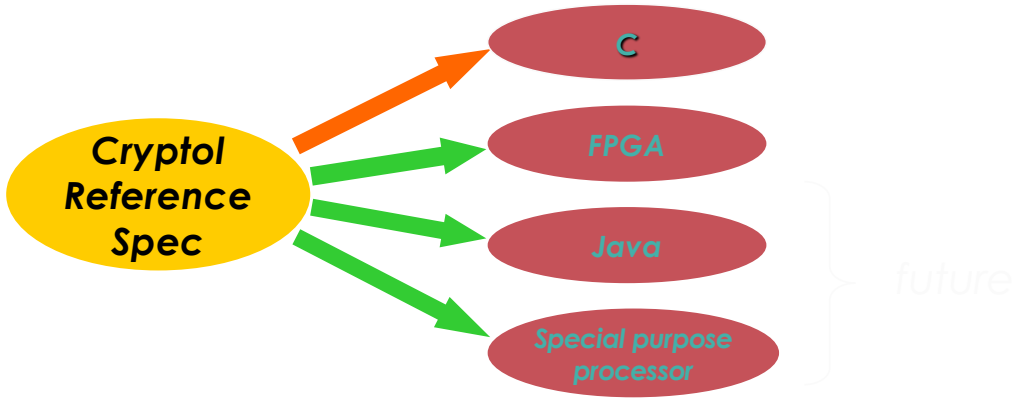
Analysis using DSL

- Symbol checking:
  - Checks non-terminal and terminal names
  - Checks indexes ($1) for validity (bounds + data)
- Type checks the productions (by type checking the action code)
- Analyses grammar for useless productions (reachability) and removes them
- Checks grammar for LALR/LR(1) conformance

Usage: Code Generation



A single correct, executable Cryptol specification can be deployed to a variety of target platforms…



- **One specification to 'get right'**
- **Many targets for use**

Tailored abstractions

- Accessible to domain experts
  - Cryptol: cryptographers
  - SQL: data analysts
- Program reliability (code size reduction)
  - PADS generates error detection code
  - ESP generates state machine context switching
- Living documentation
  - Cryptol implementations as reference specifications
  - PADS descriptions document ad hoc data formats

Caveat

- Cryptol leaves out
  - recursion to support compilation in finite space.
  - imperative variables to support mathematical reasoning.
- ESP leaves out recursive data structures and buffered channels to facilitate model checking.
- SQL and YACC restrict control flow to ensure efficient compilation.

Why not libraries?

- Some DSLs are in fact libraries
  - Example: Gradle, a language for defining build tasks.
  - Fits best in JVM-languages.
- But:
  - Complex libraries can be hard to use.
  - Type checking only at host language level.
  - More difficult to leverage domain knowledge or to generate more than one artifact.

Gradle in Android

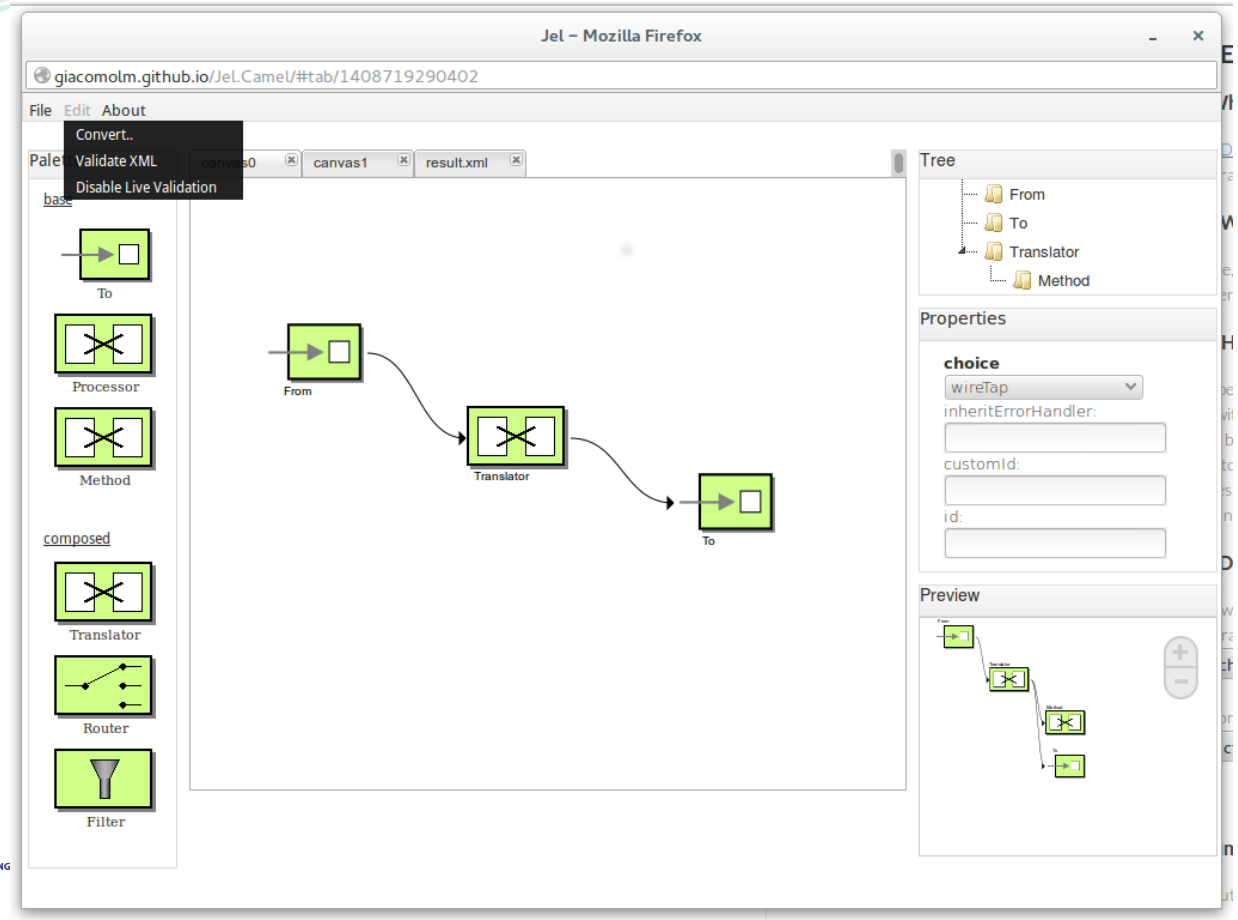
```
buildscript {
   repositories {
      mavenCentral()
   }
   dependencies {
      classpath
'com.android.tools.build:gradle:2.2.0-
beta3'
   }
}
apply plugin: 'com.android.application'

android {
   compileSdkVersion 23
   buildToolsVersion "23.1.0"
}

android {
   lintOptions {
      abortOnError false
   }
   compileSdkVersion 22
   buildToolsVersion "21.1.2"

      defaultConfig {
         targetSdkVersion 22
      }
   sourceSets {
      main {
         manifest.srcFile
'AndroidManifest.xml'
         java.srcDirs = ['src']
         resources.srcDirs = ['src']
         aidl.srcDirs = ['src']
         renderscript.srcDirs = ['src']
         res.srcDirs = ['res']
         assets.srcDirs = ['assets']
      }
      debug.setRoot('build-
types/debug')
      release.setRoot('build-
types/release')
```

DSL Script

- A language to build, configure or do in your domain.

- Not necessary textual, could be graphical
- Might reuse syntax of host language

Camel DSL


Jel – Mozilla Firefox
giacomolm.github.io/Jel.Camel/#tab/1408719290402
File
Edit
About
Convert..
Validate XML
Disable Live Validation
canvas1
result.xml
base
To
Processor
Method
composed
Translator
Router
Filter
From
Translator
To
Tree
From
To
Translator
Method
Properties
choice
wireTap
inheritErrorHandler:
customId:
id:
Preview

Types of DSLs

- External DSL
- Internal DSL

External

- DSLs that *use a different* syntax to the main language that uses them
- is a language that's parsed independently of the host general purpose language
- Examples
  - make, flex, yacc, bison
  - XPath, SQL, regexp
  - sed, awk
  - CSS, Sass
  - Cucumber/Gherkin

Gherkin for BDD

Scenario – Attempt withdrawal using stolen card.

Given I have $100 in my account.
But my card is invalid.
When I request $50.
Then my card should not be returned.
And I should be told to contact the bank.

External Pros

- Language Designer
  - Tools for compiler construction can be used
- Language User
  - Simpler to use than a GPL

External Cons

- Language Designer
  - Expensive to implement
- Language User
  - Weak tool support
  - Yet another language to learn
  - Often targeted towards a particular GPL
  - Often difficult to closely integrate with GPL

Internal

- DSLs that *share the same* syntax to the main language that uses them.
- a particular form of API in a host general purpose language. Often referred to as an interface
- Examples
  - PetitParser (Smalltalk)
  - rake, rspec (Ruby)
  - jQuery (JavaScript)
  - RPython (Python)
  - Jmock (Java)

Internal

- A subset of the host language is used.
- Popular in Java, Groovy, Lisp, Scheme, Ruby, Smalltalk and JavaScript.

Internal Pros

- Language Designer
  - No special tools required
  - No new grammar required
- Language User
  - Intermixable with GPL
  - Tools continue to work
  - No new language to learn

Internal Cons

- Limited expressivity
- Unnecessary syntactic noise
- Constrained by host language

Summary

- All languages have a domain.
- Languages provide a rich interface to computers.
- Tailored abstractions are powerful:
  - support domain experts
  - make code more reliable (shorter: boring code is generated)
- Less is more
  - Extra reasoning principles & more optimization opportunities
- Extra benefits
  - Executable, verification support, auxiliary tools, …
- Languages are constantly being designed, implemented, and used.

Personal Exercise

- Internal DSL for Graph in Java
  - https://dzone.com/articles/creating-internal-dsls-java
- External DSL for Graph using Java & ANTLR
  - https://dzone.com/articles/creating-external-dsls-using